van Emde Boas trees

Part V Advanced Data Structures

Scope

Splitting integer high/low bits supports fast predecessors and successors. Cached min/max makes the doubly logarithmic bound possible by allowing only one expensive recursion per level. Independently stored minima and linear-universe allocation determine update rules and applicability.

Prerequisites

Integer universes, bit splitting, and recursive summaries.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Compare bitmaps, summaries, and clustering.
- Define recursive cluster and summary universes.
- Derive insertion, successor, and min/max costs.
- 20.1 Preliminary approaches
- 20.2 A recursive structure
- 20.3 The van Emde Boas tree

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Preliminary approaches

The integer universe is U={0,…,u−1}, u=2^b, b≥1. successor(x) is the smallest stored value **strictly greater** than x; predecessor is strictly smaller. x need not be stored. Return NIL when absent.

Assume word-RAM with word length w≥b. Legal keys, shifts, masks, and array indexing each cost O(1). This is not bit complexity; keys spanning multiple machine words do not inherit the same constant cost.

A u-bit bitmap supports direct membership. Successor may scan forward; even with a fast first-one operation per word, worst case is Θ(u/w) words. O(1) requires the entire universe to fit one word and the model to provide the necessary bit scan. A red-black tree instead uses O(n) nodes and O(log(n+1)) time on arbitrary comparison keys without allocating u.

Clustering alone does not imply double logarithms

Split u=256 into16 groups of16 keys. Summary bit i indicates nonempty group i. Successor checks for a larger value in the current group, then finds the next nonempty group and its minimum.

Bit-by-bit scans still cost up to16 within-group steps plus16 summary steps. Group width m gives O(m+u/m), minimized near$\sqrt{u}$ at O($\sqrt{u}$), not O(log log u).

Recursing over many levels also requires counting expensive calls per level. The proto structure below shows why taking square roots of the universe alone is insufficient.

A recursive structure

Split b bits into upper ceil(b/2) and lower floor(b/2) bits:

```
upper=2^ceil(b/2); lower=2^floor(b/2)
high(x)=floor(x/lower); low(x)=x mod lower
index(h,l)=h*lower+l
```

There are upper clusters, each with universe lower. The summary has universe upper and stores nonempty cluster indices. These two sizes cannot always both be written as$\sqrt{u}$.

At u=32, b=5, upper=8, lower=4. Key27 has high6, low3, and recombines as6 • 4+3=27. Rounding$\sqrt{32}$ for both parts would lose a complete nonoverlapping bit partition.

Why proto-vEB's two recursions are slower

First let b be a power of two so splitting is exact. Without cached extrema, proto-minimum recursively finds the first nonempty group in summary and then its local minimum:

$T_{min}(u)=2T_{min}(\sqrt{u})+\Theta(1)$.

With S(b)=T(2^b), S(b)=2S(b/2)+Θ(1)=Θ(b), giving Θ(log u). Insertion that always recurses into both cluster and summary has the same bound.

Proto-successor may search its own cluster unsuccessfully, recurse into summary, then find the next group's minimum:

$T_{succ}(u)=2T_{succ}(\sqrt{u})+\Theta(\log u)$.

Substitution gives S(b)=2S(b/2)+Θ(b)=Θ(b log b), or Θ(log u log log u). Improvement must remove expensive branches, not merely count recursion depth.

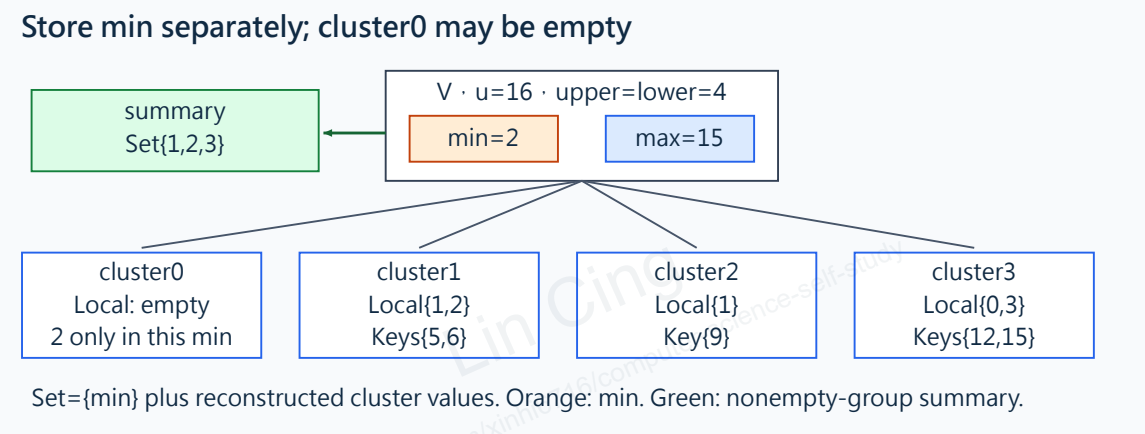
The van Emde Boas representation

Each node stores min,max, both NIL when empty. **min is excluded from all clusters.** Every other key occurs uniquely by high/low in a cluster. Summary contains exactly indices of those nonempty clusters.

A singleton has min=max with no nonempty clusters. With at least two values, max is also stored in a cluster. Base universe u=2 uses only min/max, without further clusters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The van Emde Boas representation (continued)



At u=16 and S={2,5,6,9,12,15}, min2,max15; cluster0 empty,cluster1 local{1,2},cluster2{1},cluster3{0,3},summary{1,2,3}. high(2)=0 does not force summary to contain0 because2 exists only in this node's min field.

Empty, singleton, and membership

```
member(V,x):
    if V.min==NIL: return false
    if x==V.min or x==V.max: return true
    if V.u==2: return false
    return member(V.cluster[high(x)],low(x))
minimum(V): return V.min
maximum(V): return V.max
empty-insert(V,x): V.min=V.max=x
```

Membership enters one cluster per level with strictly smaller universe. Extrema and empty/singleton tests cost O(1). empty-insert does not recursively populate the whole structure, since a new min belongs to no cluster. This constant-time base also leaves only one expensive update recursion.

The public interface validates integer $x \in 0..u-1$. It may detect duplicate insertion or absent deletion using member, return false, and leave the set unchanged. Adding one same-order query does not change asymptotic time. Internal update pseudocode assumes a new key or a present key respectively.

Mutually exclusive successor branches

```
successor(V,x):
    if V.min==NIL or x>=V.max: return NIL
    if x<V.min: return V.min
    if V.u==2: return 1
    h=high(x); l=low(x); C=V.cluster[h]
    if C.max!=NIL and l<C.max:
        return index(h,successor(C,l))
    j=successor(V.summary,h)
    if j==NIL: return NIL
    return index(j,V.cluster[j].min)
```

Reading C.max determines whether a larger same-cluster value exists. If so, recurse only there. Otherwise recurse directly into summary without first searching a cluster known to fail. Reading the next cluster's min costs O(1), so there is at most one expensive call per level.

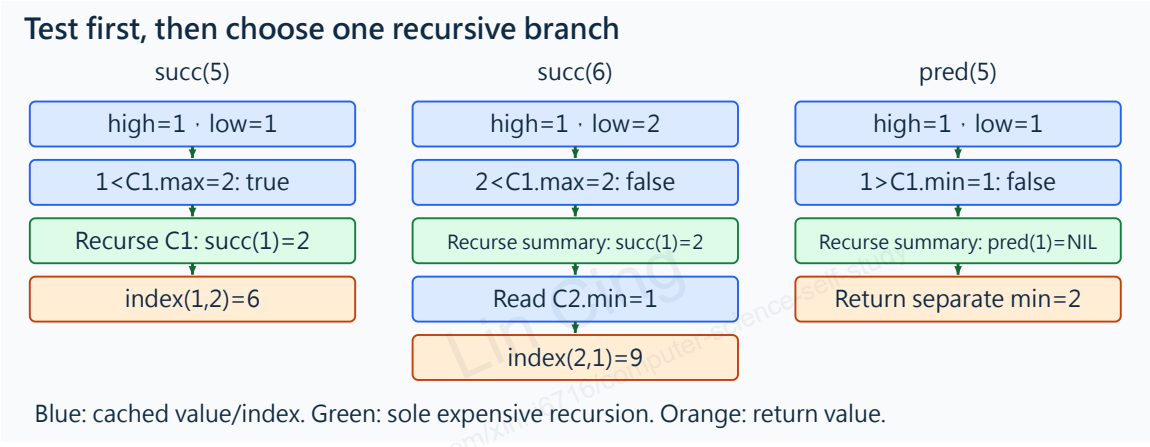
Predecessor must restore the separate minimum

```
predecessor(V,x):
    if V.min==NIL or x<=V.min: return NIL
    if x>V.max: return V.max
    if V.u==2: return 0
    h=high(x); l=low(x); C=V.cluster[h]
    if C.min!=NIL and l>C.min:
        return index(h,predecessor(C,l))
    j=predecessor(V.summary,h)
    if j==NIL: return V.min
    return index(j,V.cluster[j].max)
```

At j=NIL, earlier tests established x>V.min, so global min is a valid answer. Do not mirror successor's NIL branch mechanically: min was deliberately excluded from clusters.

For S={2,5,6,9,12,15}, pred5 has no smaller local key in cluster1 and no preceding summary group, but must still return2.

Actual same-group and cross-group queries



succ5: high1,low1,cluster1.max2. Since1<2, recurse only into cluster1, get local2, and return1・4+2=6.

succ6: high1,low2 is not below cluster1.max2, so skip that cluster. summary.succ1=2; read cluster2.min1 and return2・4+1=9.

Actual same-group and cross-group queries (continued)

Queries need not belong to S. succ3 crosses from empty cluster0 to cluster1.min1 and returns5. succ15=NIL and pred2=NIL.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Insertion swaps min and treats empty groups specially

```
insert-new(V,x):
    if V.min==NIL: empty-insert(V,x); return
    if x<V.min: swap(x,V.min)
    if V.u>2:
        h=high(x); l=low(x); C=V.cluster[h]
        if C.min==NIL:
            insert-new(V.summary,h)
            empty-insert(C,l)
        else: insert-new(C,l)
    if x>V.max: V.max=x
```

Lin Cing

A smaller new key stays in min; the old minimum becomes the value inserted into a cluster. Empty-group insertion updates two places, but cluster empty-insert is O(1), leaving only summary expensive. A nonempty group recurses only into its cluster without summary changes.

Insert1 into the example: min becomes1, old min2 enters cluster0 as local2, and summary gains0. Final max remains15. Do not also put new min1 into cluster0.

Deletion bases and promotion of a new minimum

```
delete-present(V,x):
    if V.min==V.max: V.min=V.max=NIL; return
    if V.u==2:
        V.min=V.max=(1 if x==0 else 0); return
    if x==V.min:
        h=V.summary.min
        x=index(h,V.cluster[h].min)
        V.min=x
    h=high(x); l=low(x); C=V.cluster[h]
    delete-present(C,l)
```

Continue with the next slide's summary/max cleanup. In a multi-element set, deleting min promotes the smallest clustered key, then removes that promoted value from its old cluster to preserve exclusion. Variable x now denotes the promoted value, not the caller's original deleted minimum.

Complete summary and maximum cleanup

After the preceding deletion in C:

```
if C.min==NIL:
    delete-present(V.summary,h)
    if x==V.max:
        j=V.summary.max
        if j==NIL: V.max=V.min
        else: V.max=index(j,V.cluster[j].max)
elif x==V.max:
    V.max=index(h,C.max)
```

An emptied C had only one element before deletion, so its deletion used O(1) base work before summary recursion. If cluster deletion really recursed through several levels, C cannot become empty and no summary recursion follows. The two expensive branches remain exclusive.

A stale summary can direct later queries into empty groups. A stale max can make successor choose an incorrect branch. These fields are required invariants, not optional hints.

Recompute updates from the same data

Starting at S={2,5,6,9,12,15}:

After	min	max	cluster0	cluster1	cluster2	cluster3	summary
Initial	2	15	Empty	{1,2}	{1}	{0,3}	{1,2,3}
insert1	1	15	{2}	{1,2}	{1}	{0,3}	{0,1,2,3}
delete1	2	15	Empty	{1,2}	{1}	{0,3}	{1,2,3}
delete15	2	12	Empty	{1,2}	{1}	{0}	{1,2,3}

Deleting1 promotes2 from cluster0, then deletes local2 and removes newly empty group0 from summary. Deleting15 leaves local0 in cluster3, so summary retains3 and new max=3・4+0=12. Combining min with all clusters reconstructs every row's complete set without lost or duplicate promoted values.

Correctness and doubly logarithmic time

Induct on b. At u=2, min/max represent empty, singleton, or{0,1} exactly. Assume operations correct on smaller universes. Distinct high ranges partition values, index inverts high/low, and min is stored separately.

Membership enters its only possible group. Successor/predecessor use local extrema to test same-group existence, otherwise summary finds the nearest group, with separate min handling. Insertion swaps and deletion promotions transfer exactly one representative; summary stays equivalent to nonempty clusters. Thus representation and set semantics are preserved.

Each operation has at most one nonconstant subcall on at most ceil(b/2) bits:

$T(2^b) \le T(2^{\lceil b/2 \rceil}) + O(1)$.

For $b \ge 2$,$\lceil b/2 \rceil \le 2b/3$, so reaching b=1 needs O(log b) levels, giving O(1+log log u). This depends on the proved exclusive branches, not only structural depth.

Preallocated space recurrence

The original version builds every summary/cluster in advance. Even an empty set uses Θ(u) words and Θ(u) initialization time. Including node fields and cluster-pointer arrays, for u>2,

$$P(2^b)=2^{\lceil b/2\rceil}P(2^{\lfloor b/2\rfloor})+P(2^{\lceil b/2\rceil})+\Theta(2^{\lceil b/2\rceil}).$$

Let F(b)=P(2^b)/2^b and ε_b=2^(−floor(b/2)). For a fixed c,

$$F(b)\le F(\lfloor b/2\rfloor)+\varepsilon_b\ F(\lceil b/2\rceil)+c\ \varepsilon_b.$$

With G=F+c,

$$G(b)\le(1+\varepsilon_b)\max(G(\lfloor b/2\rfloor),G(\lceil b/2\rceil)).$$

Follow the larger term down to b=1 and bound the accumulated finite product, retaining summary cost rather than ignoring it.

Linear space and allocation limitations

Counting r levels backward along that halving path, bit sizes are at least2^(r−1)+1. Corresponding ε values are bounded by1/2,1/2,1/4,1/16,…,whose sum is bounded independently of b. Since1+x≤e^x,

$$\Pi(1+\varepsilon)\le\exp(\Sigma\varepsilon)=O(1).$$

Thus F(b)=O(1),P(u)=O(u). Cluster branches alone partition the original universe into u/2 base nodes at u=2, giving Ω(u), hence Θ(u). Initializing these fields also costs Θ(u).

For huge u and small n, preallocation can be unattractive. On-demand clusters and hash dictionaries require separate proofs of construction,dictionary time,and space. Calling allocation lazy does not establish O(n) space and worst-case O(log log u) operations simultaneously. The proved version here uses preallocated arrays and direct indexing on word-RAM.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter20.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study